

MANAGERIAL EXECUTION PLAN

PROJECT VISION

The goal is to deliver an end-to-end MVP prediction system that ingests nightly stats & sentiment, enforces a locked feature contract, runs a deterministic candidate pool, and surfaces consistent rankings via API or dashboards. Major decisions are already locked: we keep the 24-feature vector, score the top 30 players by minutes, normalize inputs with pre-computed scaler parameters, and expect Python 3.12 when loading the packaged MLflow model.

KEY DELIVERABLES

1. **Feature Contract (WS1)**: Maintain `feature_schema_v1.json` and scaler artifacts so every inference receives the same 24 numbers in the exact order the model was trained on.
2. **Hybrid Stats + Sentiment Pipeline (WS2 & WS3)**: Nightly loaders pull Basketball-Reference data plus curated sentiment ratings, compute advanced metrics (PER, BPM, Win Shares, pace, etc.), and annotate provenance so we know whether a feature is derived or sourced.
3. **Runtime Feature Builder (WS4)**: Build named feature records per candidate, validate them against the schema, vectorize into `(N, d)`, and normalize using the locked scaler.
4. **API & Ranking (WS5)**: Serve both the legacy `POST /predict` and the new `GET /candidate_pool` endpoint that returns the latest ranked list, candidate metadata, and the deterministic snapshot definition.
5. **Testing & QA (WS6)**: Keep unit and integration tests around schema validation, feature pipeline outputs, and ranking logic so regressions are caught early.
6. **Documentation & Runbooks (WS7)**: README, agent checklist, and this execution plan capture the tooling, cron cadence, Python dependency notes, and gating log so reviewers know what to verify.

OPERATIONAL WORKFLOW

- **Nightly job**: Run `scripts/build_candidate_pool_vectors.py` inside the Python 3.12 sandbox. It rebuilds the scaler, normalizes the top-30 vectors, records the top-N ranking (default N=5), and writes both the ranking payload and the vector set under `data_exporters/candidate_pool/` along with `json/scaler_params_v1.json`.
- **Candidate pool API**: `GET /candidate_pool` accepts optional query parameters (`top_n`, `pool_size`, `season`, `mode`) and returns a JSON payload describing the pool definition, snapshot timestamp, and sorted probabilities. This ensures downstream teams always fetch the same deterministic set that the nightly job produced.
- **Cron & logging**: Each pipeline run is noted in `progress.log` and mirrored

in ``docs/agent-checklist.md`` so the gating-ready narrative (instrumentation checks + candidate pool reruns) remains traceable. The checklist is the single source for reviewers when verifying the next handoff.

RISK NOTES

- The MLflow artifact uses torch objects tied to Python 3.12. Running the ranking script or endpoint under any other interpreter raises a ``TypeError: code() takes at most 16 arguments``. Mitigation: keep the entire export/serving flow inside the ``.venv312`` sandbox or rebuild the artifact under the target interpreter.
- If we ever retrain the model or adjust the schema, reissue `feature_schema_v1.json`` and the scaler (mean + scale arrays) so the normalization contract remains valid.

NEXT STEPS FOR REVIEW

1. Confirm the nightly schedule targets the Python 3.12 sandbox and retains the exported artifacts for auditors.
2. Point dashboards or gating scripts at `GET /candidate_pool`` so they consume the same normalized ranking data.
3. Continue using the checklist + plan to document every instrumentation check or rerun so reviewers can trace the verification story without digging through logs.